

1. Overview of Intermediate Representations (IR)

Intermediate Representation (IR) is an abstract machine representation positioned between the front-end (lexical analysis, parsing, semantic analysis) and the back-end (instruction selection, register allocation, machine code generation) of a compiler. Using an IR provides machine independence: targeting a new CPU architecture requires only rewriting the back-end without modifying lexical or parsing stages.

2. Key Definitions and Structures

- Three-Address Code (TAC): An abstract language where every instruction contains at most one operator on the right-hand side and at most three address operands. Canonical form: $x = y \text{ op } z$, or unary $x = \text{op } y$.
- Quadruples: A record structure representation of TAC with four distinct fields: (op, arg1, arg2, result). For example, $t1 = a + b$ is stored as (+, a, b, t1).
- Triples: A representation avoiding explicit temporary variable names by referencing temporary results by their statement index array position: (op, arg1, arg2).
- Abstract Syntax Tree (AST): A condensed hierarchical tree representation of source syntax where internal nodes represent operators and leaf nodes represent operands.

3. Translation of Expressions and Equations

Consider the high-level arithmetic expression: $\text{result} = (a + b) * (c - d) + (a + b)$.

In Three-Address Code with Common Subexpression Elimination (CSE):

Step 1: Compute temporary addition $t1 = a + b$

Step 2: Compute temporary subtraction $t2 = c - d$

Step 3: Compute multiplication $t3 = t1 * t2$

Step 4: Reuse previously computed t1 for final addition $t4 = t3 + t1$

Step 5: Assign final value $\text{result} = t4$

This translation reduces total arithmetic operations from four down to three through DAG reuse.

4. Control Flow and Boolean Expressions

Control flow statements (such as if-then-else and while loops) are translated into conditional

jumps 'if condition goto L1' and unconditional jumps 'goto L2'. Backpatching is a single-pass

technique where target branch labels are left empty during initial code emission and filled in

once target statement addresses become known.